Classification: supervised

Pritha Banerjee

University of Calcutta

banerjee.pritha74@gmail.com

April 28, 2026

Overview

Overview

Regression: Estimation of f

- Given p predictors(observations) $X_1, X_2, ..., X_p$ and response Y the relationship is given by $Y = f(X) + \epsilon$
- Either **predict** $\hat{Y}$ using parametric or non parametric methods or **infer**, that is, the relationship between predictors and response variable
- Use less flexible models for inference and more flexible model for accurate predictions.

Regression Vs. Classification

- Responses Y may be a) quantitative (numerical) b) qualitative(categorical, ie., values in one of k classes)
- Typically problems with a **quantitative response** are considered **regression problems**, while those involving a **qualitative response** are often referred to as **classification problems**.
- However, the distinction is not very crisp.
- Least squares linear regression is used with a quantitative response, whereas **logistic regression** is typically used with a **qualitative** (two-class, or binary) response (often used as a classification method). But, since it estimates class probabilities, it can be thought of as a regression method as well.
- Some statistical methods, such as K-nearest neighbors and boosting (typically classification method) can be used for either quantitative or qualitative responses.
- However, whether the predictors are qualitative or quantitative is generally considered less important.

Supervised Vs. Unsupervised Learning

- Simple linear regression model is **Supervised Learning** since, response y is predicted or inferred using SLR model built on the predictor and response measurements.
- Logistic regression, Support Vector Machine are examples of Supervised Learning
- **Unsupervised Learning** describes situation in which for every observation $i = 1, \cdots, n$ there is a vector of measurements x_i but no associated response y_i; not possible to fit a linear regression model, since there is no response variable.
- Solution for unsupervised Learning is **Clustering** or **Cluster Analysis**
- The goal of cluster analysis /clustering is to ascertain, on the basis of $x_1, x_2, \cdots x_n$, whether the observations fall into relatively distinct groups.

Overview

Error Rate: Classification

- Suppose wa want to estimate f from training observations $\{(x_1, y_1), \cdots, (x_n, y_n)\}$, where y_i s are **qualitative**. To **quantifying the accuracy** of $\hat{f}$ is **training error rate**, compute the proportion of mistakes that are made if we apply our estimate $\hat{f}$ to the training observations,

$$\frac{1}{n}\sum_{i=1}^{n} I(y_i \neq \hat{y}_i)$$

  $I(y_i \neq \hat{y}_i)$ is an **indicator variable** that equals 1, if $y_i \neq \hat{y}_i$ else, 0

- **test error rate:** Error rates that result from applying the classifier to test observations that were not used in training. Test error rate associated with a set of test observations of the form (x_0, y_0) is $Avg(I(y_0 \neq \hat{y}_0))$ where $\hat{y}_0$ is the **predicted class label** that results from applying the classifier to the test observation with predictor x_0. A **good classifier** is one for which the **test error is smallest**.

Statistical Decision Theory: Quantitative Output

- Let $X \in \mathbb{R}^p$ and $Y \in \mathbb{R}$ denote real valued random input and output vectors respectively with joint probability distribution $Pr(X, Y)$.
- Determine function $Y = f(X)$ minimizing the **loss function** $L(Y, f(X)) = (Y - f(X))^2$. The Expected Prediction Error, $EPE(f) = E(Y - f(X))^2 = \int (y - f(x)) Pr(dx, dy)$
- By conditioning on X, $EPE(f) = E_X E_{Y|X}(Y - f(X))^2|X)$
- Minimize EPE pointwise: $\hat{f}(x) = \arg\min_c E_{Y|X}((Y - c)^2|X = x)$;
- The solution is $\hat{f}(x) = E(Y|X = x)$; the best prediction of Y at any point $X = x$ is the **conditional mean**, when **best** is measured by **average squared error**.

Statistical Decision Theory: Qualitative Output

- When the output is a categorical (qualitative) variable G, Loss function is represented as $K \times K$ matrix L, K is the number of class or categorical values. $L(k, l) = 0$, when $k = l$ (diagonal), else, $L(k, l)$ is the cost of misclassifying class k as l. Each misclassification has unit cost (zero-one loss function).
- Expected Prediction Error, $EPE = E(L(G, \hat{G}(X)))$; after conditioning, $EPE = E_X \sum_{k=1}^{K} L(G_k, \hat{G}(X)) Pr(G_k|X)$
- Minimize EPE pointwise:
  $\hat{G}(x) = \arg\min_{g \in G} \sum_{k=1}^{K} L(G_k, g) Pr(G_k|X = x)$; since it is $0-1$ loss function, $\hat{G}(x) = \arg\min_{g \in G}(1 - Pr(g|X = x))$
- Simplifying, $\hat{G}(x) = G_k$ if $Pr(G_k|X = x) = \max Pr(g|X = x)$.
- Using Bayes classifier, we classify to the most probable class, using the conditional (discrete) distribution $Pr(G|X)$

Bayes Classifier

- The **test error rate** is **minimized, on average,** by a very simple classifier that assigns each observation to the **most likely class, given its predictor values**. Assign a test observation with predictor vector x_0 to the class j for which $Pr(Y = j|X = x_0)$ is **largest**
- For a two class problem, Bayes classifier corresponds to predicting class **one** if $Pr(Y = 1|X = x0) > 0.5$, and class **two** otherwise.
- The points where the probability is exactly 50% is called the **Bayes decision boundary**.
- Bayes classifier produces the **lowest possible test error rate**, called the **Bayes error rate**.
- Since the Bayes classifier always chooses the class for which $Pr(Y = j|X = x_0)$ is **largest**, the **error rate** at $X = x_0$ will be $1 - max_j Pr(Y = j|X = x_0)$. Overall Bayes error rate is $1 - E(max_j Pr(Y = j|X))$. It is the irreducible error.

Example

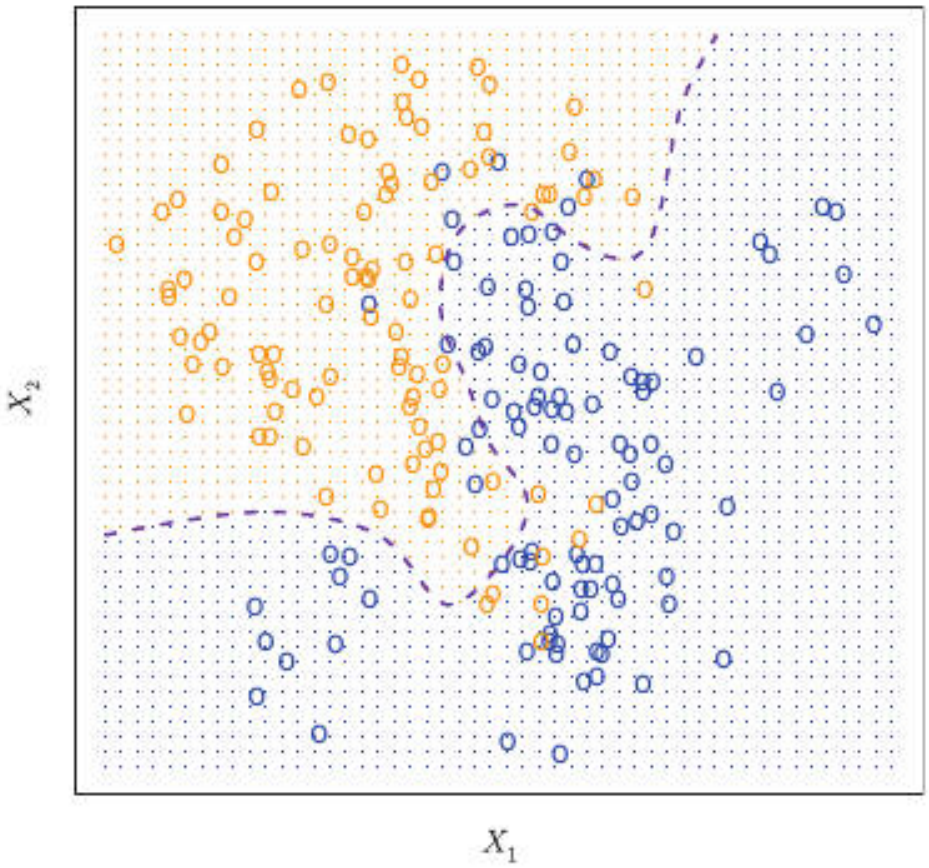


FIGURE 2.13. *A simulated data set consisting of* 100 *observations in each of two groups, indicated in blue and in orange. The purple dashed line represents the Bayes decision boundary. The orange background grid indicates the region in which a test observation will be assigned to the orange class, and the blue background grid indicates the region in which a test observation will be assigned to the blue class.*

K Nearest Neighbour (KNN)

- Theoretically we would always like to predict **qualitative responses** using the Bayes classifier, but we do not know the conditional distribution of Y given X, and thus it only serves as an unattainable gold standard against which to compare other methods.
- **KNN:** Given a positive integer K and a test observation x_0 , the KNN classifier first identifies the K points in the **training data** that are **closest to** x_0 , represented by N_0.
- Then it estimates the **conditional probability** for class j as the fraction of points in N_0 whose response values equal j:

$$Pr(Y = j|X = x_0) = \frac{1}{K} \sum_{i \in N_0} I(y_i = j)$$

- Finally, KNN applies Bayes rule and classifies the test observation x_0 to the class with the **largest probability**.

K nearest neighbour (KNN): Properties

- K-NN is a **non-parametric, supervised, multiclass** classification algorithm, whereas logistic regression is parametric algorithm.
- K, the number of neighbours of a given point x to be chosen around, is the tuning parameter and not derived from data
- K NN is called **lazy** learning algorithm since the computations are deferred until classification; No explicit training, generalization.
- The function is approximated locally, unlike logistic regression
- Typically used when decision boundaries are **non-linear** and there are large volumes of data
- Inputs are quantitative or qualitative, output is categorical value, the class.
- K NN could also be used for function approximation problems (prediction); by averaging the y values of the K neighbours, whereas for classfication it uses majority voting of K neighbours of input x
- No assumption about data distribution, linear seperability (non parametric)
- Need to find the best K value by running the K-NN algorithm for increasing K values.

K nearest neighbour (KNN): Algorithm

To define K neighbours of a given point x, different distant metric can be used to find the distance between x and the points is the dataset; such as Euclidean distance, Mahalanobis distance, Hamming distnce etc.
Algorithm:

- Compute chosen distance metric between the test data x and all labelled data points($O(nd)$), where d is the number of features.
- Select the closest K labelled data points of x (K neighbours, $O(nK)$, using extra space and without sorting)
- Find the class label that the majority of the K neighbours have and assign that label to x. ($O(K)$)

Time Complexity : $O(nd + Kn)$

K nearest neighbour (KNN): Issues

- The best choice of K depends on data
- Larger $K > 20$ reuces effect of noise on classification, thus **underfitting**, leading to low variance, high bias (**underfitting**), but robust to outliers; may overlook local patterns; good for large dataset with many outliers
- Smaller $K = 1$ or $K = 3$ tend to be affected by noise (**overfitting**), produces complex, flexible boundaries; lower bias, high variance (**overfitting**); use for datasets with clear, local patterns
- $K = 5$ to $K = 10$ is balance, usually K is taken to be **odd**.
- Must remove irrelevant, redundant features

K nearest neighbour (KNN): Merits

- Simple and Intuitive: Easy to understand and implement.
- No Training Phase (Lazy Learner): It does not build a model, which makes training fast, as it simply stores the dataset.
- Non-parametric: No assumptions are made about the underlying data distribution, making it flexible for non-linear data.
- Versatile: Can be used for both classification and regression tasks.

K nearest neighbour (KNN): Demerits

- High Computational Cost: Since it calculates the distance between a new point and all training points, it becomes very slow with large datasets.
- Memory Intensive: Stores the entire training dataset to make predictions.
- Curse of Dimensionality: Performance degrades significantly as the number of features (dimensions) increases, as distances become less meaningful; dimensionality reduction is required.
- Sensitive to Noise and Outliers: Noisy data can skew the results, as KNN relies on local, rather than global, data points.
- Requires Feature Scaling: Normalization or standardization is essential to prevent features with larger magnitudes from dominating the distance metric

Use KNN: when the dataset is small and noisy, when a quick, interpretable solution is required, when the data does not conform to traditional linear assumptions.

Choosing K

- **k-fold Cross-Validation:** divide the dataset into k parts. The model is trained on some of these parts and tested on the remaining ones. This process is repeated for each part. The k value that gives the highest average accuracy during these tests is usually the best one to use.
- **Elbow Method:** Draw a graph showing the error rate or accuracy for different k values. As k increases the error usually drops at first. But after a certain point error stops decreasing quickly. The point where the curve changes direction and looks like an "elbow" is usually the best choice for k.

KNN Approach: Example

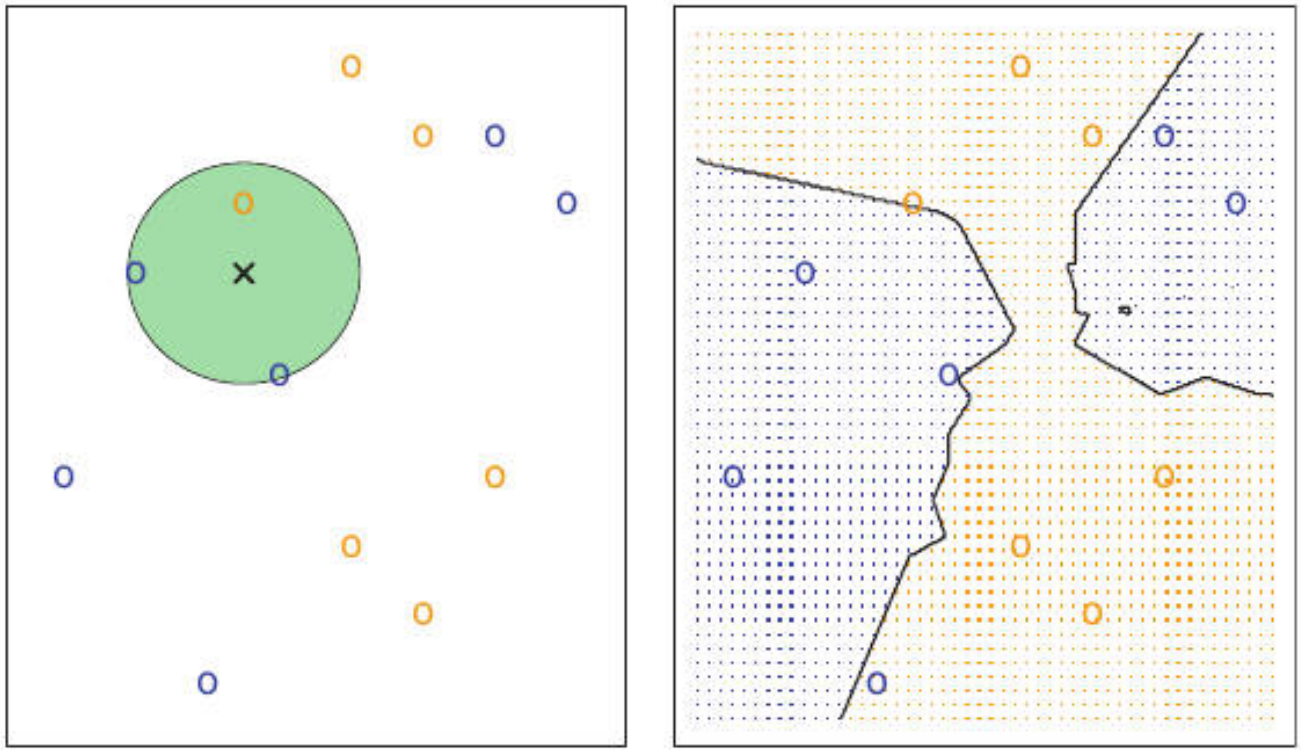


FIGURE 2.14. *The KNN approach, using $K = 3$, is illustrated in a simple situation with six blue observations and six orange observations.* Left: *a test observation at which a predicted class label is desired is shown as a black cross. The three closest points to the test observation are identified, and it is predicted that the test observation belongs to the most commonly-occurring class, in this case blue.* Right: *The KNN decision boundary for this example is shown in black. The blue grid indicates the region in which a test observation will be assigned to the blue class, and the orange grid indicates the region in which it will be assigned to the orange class.*

KNN and Bayesian Decision Boundary for $K = 10$

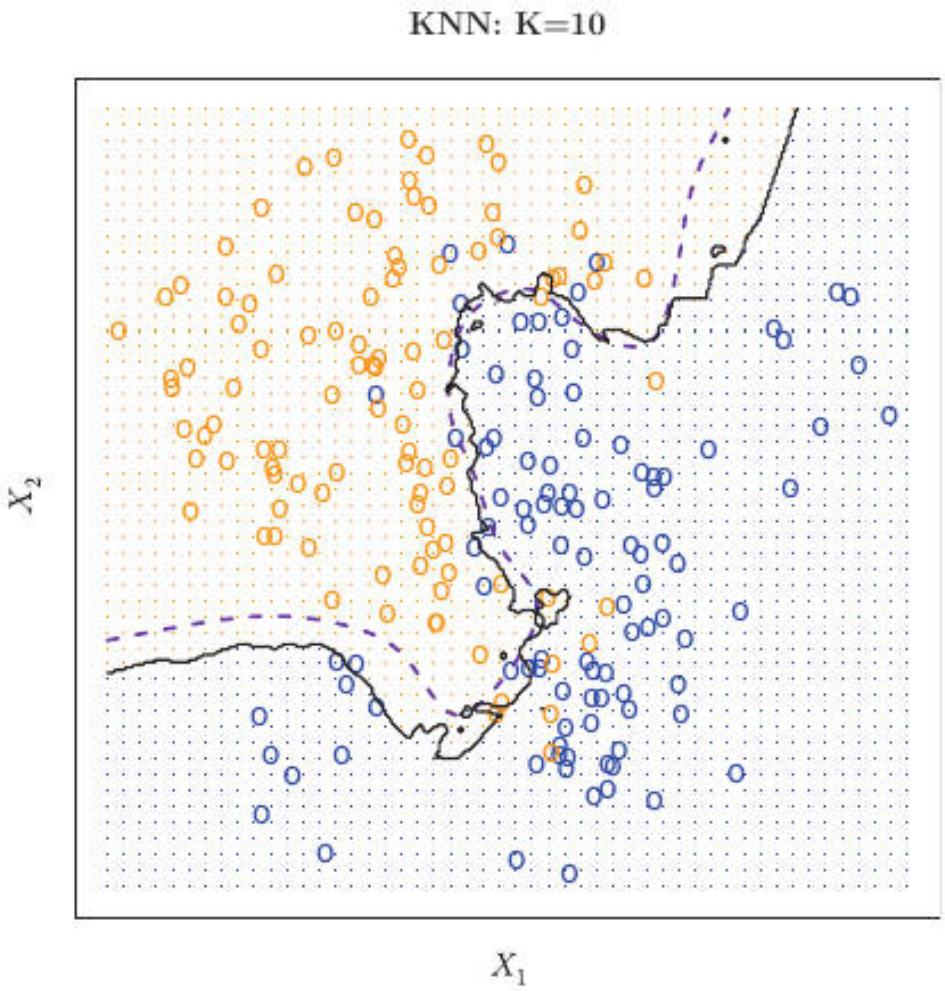


FIGURE 2.15. *The black curve indicates the KNN decision boundary on the data from Figure 2.13, using $K = 10$. The Bayes decision boundary is shown as a purple dashed line. The KNN and Bayes decision boundaries are very similar.*

KNN and Bayesian Decision Boundary for $K = 1, 100$

- $K = 1$: decision boundary is overly flexible and does not correspond to Bayes decision boundary,i.e, **low bias but high variance (overfitting)**.
- As K grows, it becomes **less flexible** and produces a decision boundary close to **linear**. i.e, **high bias (underfitting) and low variance** classifier
- Test error rates are 0.1695 and 0.1925, for $K = 1$ and 100 respectively, that suggest bad predictions.

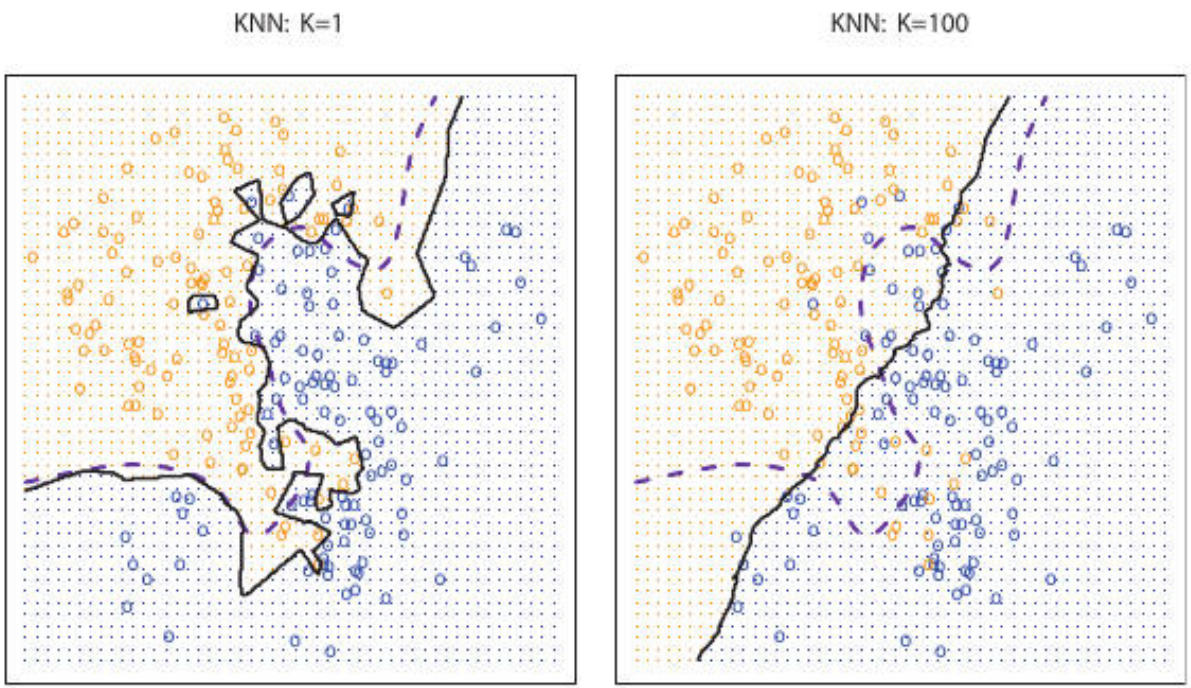
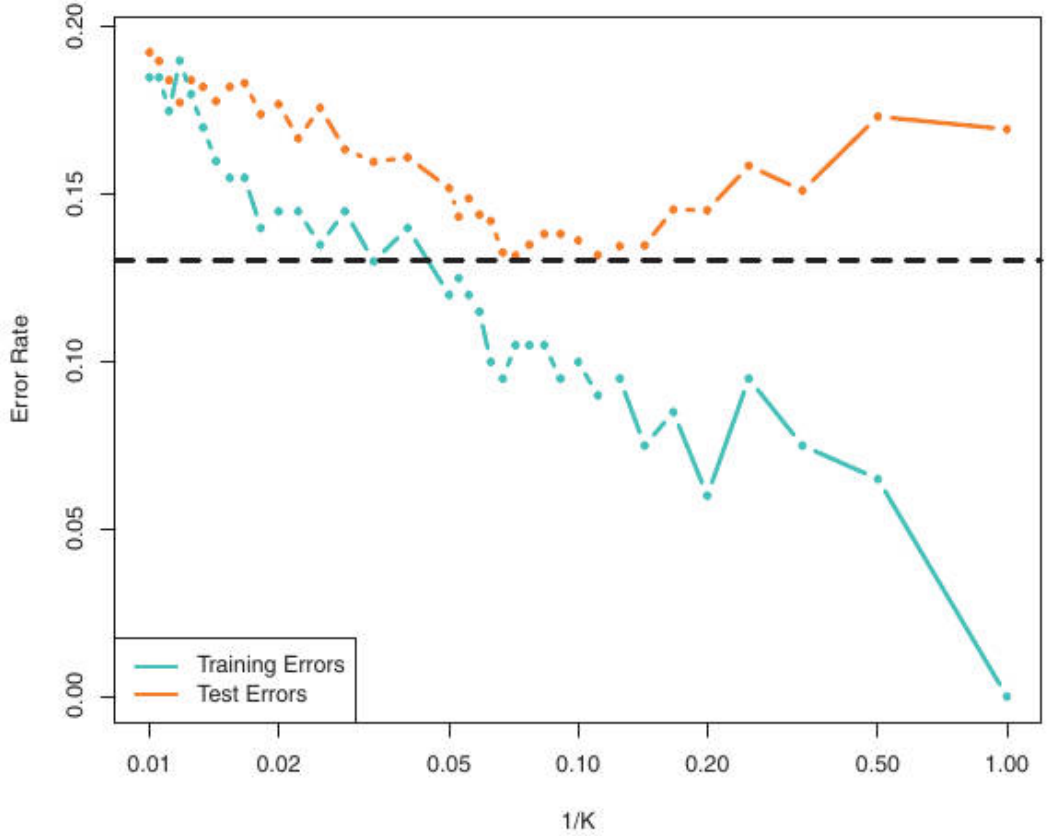


FIGURE 2.16. *A comparison of the KNN decision boundaries (solid black curves) obtained using $K = 1$ and $K = 100$ on the data from Figure 2.13. With $K = 1$, the decision boundary is overly flexible, while with $K = 100$ it is not sufficiently flexible. The Bayes decision boundary is shown as a purple dashed*

KNN Training and Test error rate

- There is no strong relationship between the training error rate and the test error rate.
- With $K = 1$ (flexible), KNN **training error rate** is 0, but **test error rate** may be quite **high**.
- KNN test and training errors plotted as a function of $1/K$. As $1/K$ increases, the method becomes more flexible.
- As in regression, the training error rate consistently declines as the flexibility increases. However, the test error exhibits a characteristic U-shape, declining at first (with a minimum at approximately $K = 10$) before increasing again when the method becomes excessively flexible and **overfits**.



Overview

Linear Method for classification

- **Categorical response** Y can be properly **coded to integers** so that **linear regression** can be used for prediction in the context of classification problem. For example, in a two class problem, response Y can be coded as 0 and 1. if $\hat{y} > 0.5$, we predict class 1 else class 2. However for multiclass problem, this approach may not give good results.
- Suppose there are K classes, labeled $1, 2, \cdots, K$, and the fitted **linear model** for the k^{th} indicator response variable is $\hat{f}_k(x) = \hat{\beta}_{k0} + \hat{\beta}_k^T x$.
- The **decision boundary** between class k and l is that set of points for which $\hat{f}_k(x) = \hat{f}_l(x)$, that is, the set $\{x : (\hat{\beta}_{k0} - \hat{\beta}_{l0}) + (\hat{\beta}_k - \hat{\beta}_l)^T x = 0\}$, an affine set or hyperplane. Thus, for any pair of classes, the **input space is divided into regions of constant classification**, with piecewise hyperplanar decision boundaries.
- This regression approach is a member of a class of methods that model **discriminant functions** $\delta_k(x)$ for **each class**, and then classify x to the class with the **largest value** for its **discriminant function**; models posterior probabilities $Pr(G = k|X = x)$.

Classification: Decision Boundaries

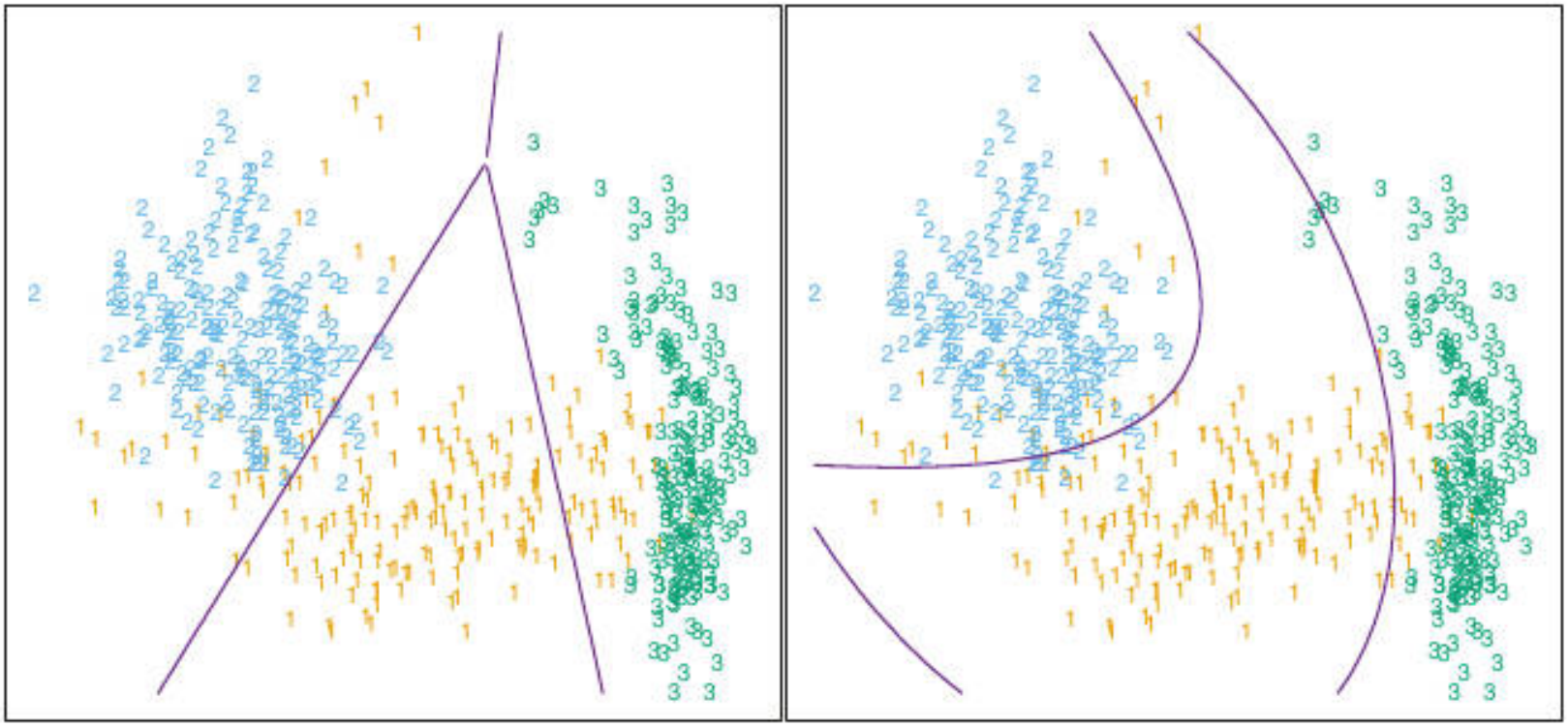


FIGURE 4.1. *The left plot shows some data from three classes, with linear decision boundaries found by linear discriminant analysis. The right plot shows quadratic decision boundaries. These were obtained by finding linear boundaries in the five-dimensional space* $X_1, X_2, X_1X_2, X_1^2, X_2^2$. *Linear inequalities in this space are quadratic inequalities in the original space.*

Linear Regression using indicator varibles

- If G has K classes, K indicator variable Y_k, $k = 1, \cdots, K$, with $Y_k = 1$ if $G = k$ else 0. Let $Y = (Y_1, \cdots, Y_K)$, N training instances of these form an NK indicator response matrix Y. Y is a matrix of 0's and 1's, with each row having a single 1.
- Fit a linear regression model to each of the columns of Y simultaneously by $\hat{Y} = X(X^T X)^{-1} X^T Y$.
- We have a **coefficient vector for each response column** y_k , and hence a $(p+1) \times K$ coefficient matrix $\hat{\beta} = (X^T X)^{-1} X^T Y$. Here X is the model matrix with $p+1$ columns corresponding to the p inputs, and a leading column of 1's for the **intercept**.
- A new observation with input x is classified as follows:
  - compute the fitted output $\hat{f}(x)^T = (1, x^T)\hat{B}$, a K vector;
  - identify the **largest component** and classify accordingly: $\hat{G}(x) = \arg\max_{k \in G} \hat{f}_k(x)$.
- For the random variable Y_k , $E(Y_k|X = x) = Pr(G = k|X = x)$, so conditional expectation of each of the Y_k seems a sensible goal. Is $\hat{f}_k(x)$ reasonable **estimates** of the posterior probabilities $Pr(G = k|X = x)$?

Another approach

- $\hat{f}_k(x)$ can be negative or greater than 1, and thus can not predict the class appropriately as it is.
- **Alternative :** Construct targets t_k for each class, where t_k is the k^{th} column of the KK identity matrix.
- Reproduce the appropriate target for an observation as follows. With the same coding as before, the response vector y_i (i^{th} row of Y) for observation i has the value $y_i = t_k$ if $g_i = k$.
- Fit the linear model by least squares:$\min_B \sum_{i=1}^{N} ||y_i - \left[(1, x_i^T)B\right]^T||^2$
- A new observation is classified by computing its fitted vector $\hat{f}(x)$ and classifying to the closest target: $\hat{G}(x) = \arg\min_k ||\hat{f}(x) - t_k||^2$.

Problem of linear method

For $K \geq 3$, linear method masks the class in the middle of two classes.

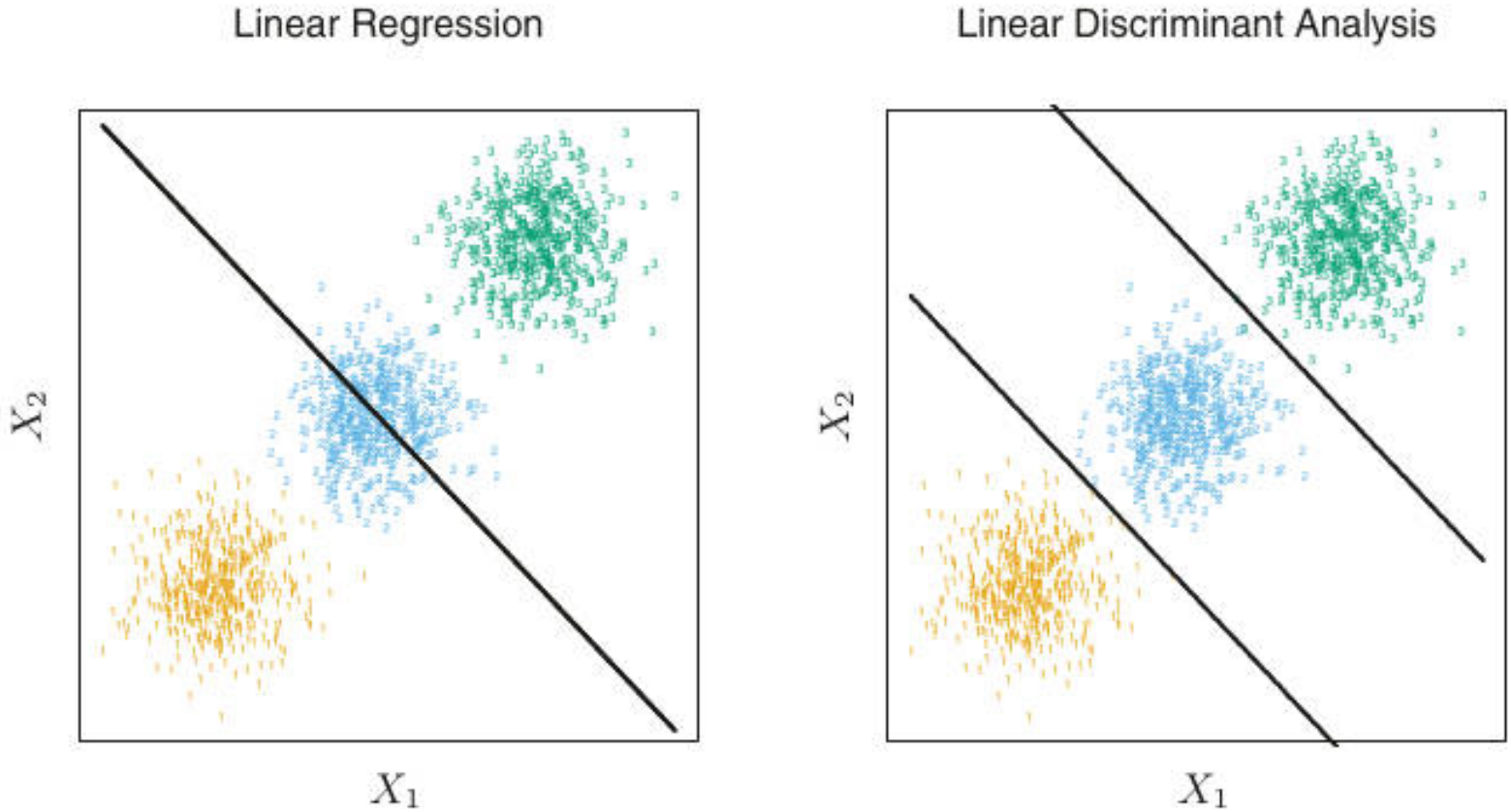


FIGURE 4.2. *The data come from three classes in* $\mathbb{R}^2$ *and are easily separated by linear decision boundaries. The right plot shows the boundaries found by linear discriminant analysis. The left plot shows the boundaries found by linear regression of the indicator response variables. The middle class is completely masked (never dominates).*